

TIER 2 — ADVANCED USER INTERACTIONS

ASSIGNMENT 4 — JavaScript Alerts and Confirms

1. Experiment Title

Handling JavaScript Alerts, Confirm Boxes and Prompt Boxes Using Selenium

2. Problem Statement

Automate JavaScript-based pop-ups on a practice web page using Selenium WebDriver. The automation should handle an Alert, Confirm Box, and Prompt Box according to the specified actions.

3. Objective

- To handle JavaScript Alert boxes.
- To accept an Alert.
- To dismiss a Confirm Box.
- To interact with a Prompt Box.
- To enter text into a Prompt using `driver.switch_to.alert.send_keys()`.

4. Tools, Software and Concepts Used

Tools & Software:

- Python
- Selenium WebDriver
- Google Chrome
- VS Code / PyCharm

Concepts:

- JavaScript Alert
- Confirm Box
- Prompt Box
- `switch_to.alert`
- `accept()`

- dismiss()
- send_keys()
- Selenium WebDriver

5. Source Code / Implementation Steps

```
from selenium import webdriver

from selenium.webdriver.common.by import By
import time

def test_javascript_alerts():

    driver = webdriver.Chrome()

    driver.maximize_window()

    driver.get("https://the-internet.herokuapp.com/javascript_alerts")

    driver.find_element(

        By.XPATH, "//button[text()='Click for JS Alert']"

    ).click()

    alert = driver.switch_to.alert

    print("Alert text:", alert.text)

    alert.accept()

    driver.find_element(

        By.XPATH, "//button[text()='Click for JS Confirm']"

    ).click()

    confirm = driver.switch_to.alert

    print("Confirm text:", confirm.text)

    confirm.dismiss()

    driver.find_element(

        By.XPATH, "//button[text()='Click for JS Prompt']"

    ).click()
```

```

prompt = driver.switch_to.alert
print("Prompt text:", prompt.text)

prompt.send_keys("Selenium Assignment 4")

prompt.accept()

result = driver.find_element(By.ID, "result").text

assert "Selenium Assignment 4" in result

print("Prompt result:", result)

time.sleep(10)

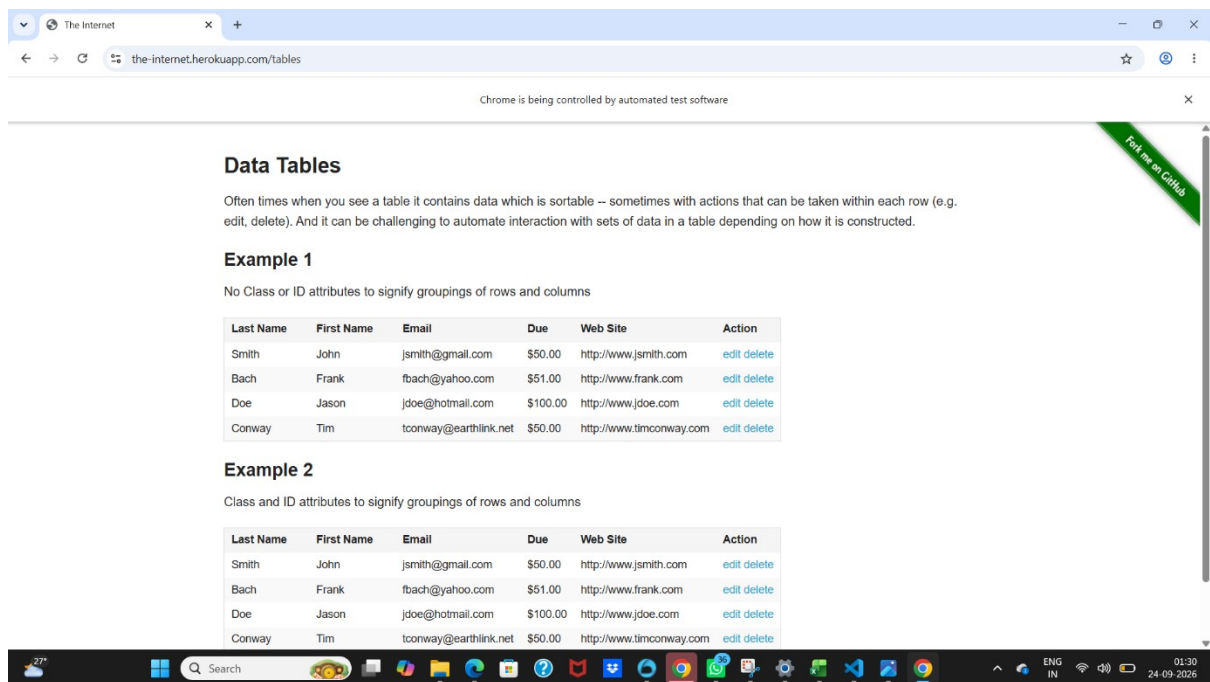
driver.quit()

```

Execution

python test_assignment4.py

6. Output with Clearly Labelled Screenshots



the-internet.herokuapp.com/tables

Ask Gemini

Test me on Celsius

Data Tables

Often times when you see a table it contains data which is sortable -- sometimes with actions that can be taken within each row (e.g. edit, delete). And it can be challenging to automate interaction with sets of data in a table depending on how it is constructed.

Example 1

No Class or ID attributes to signify groupings of rows and columns

Last Name	First Name	Email	Due	Web Site	Action
Smith	John	jsmith@gmail.com	\$50.00	http://www.jsmith.com	edit delete
Bach	Frank	fbach@yahoo.com	\$51.00	http://www.frank.com	edit delete
Doe	Jason	jdoe@hotmail.com	\$100.00	http://www.jdoe.com	edit delete
Conway	Tim	tconway@earthlink.net	\$50.00	http://www.timconway.com	edit delete

Example 2

Class and ID attributes to signify groupings of rows and columns

Last Name	First Name	Email	Due	Web Site	Action
Smith	John	jsmith@gmail.com	\$50.00	http://www.jsmith.com	edit delete
Bach	Frank	fbach@yahoo.com	\$51.00	http://www.frank.com	edit delete
Doe	Jason	jdoe@hotmail.com	\$100.00	http://www.jdoe.com	edit delete
Conway	Tim	tconway@earthlink.net	\$50.00	http://www.timconway.com	edit delete

SELENIUM ASSIGNMENTS

test_assignment4.py

```
def test_javascript_alerts():
    # Enter text into prompt
    prompt.send_keys("Selenium Assignment 4")

    # Submit prompt
    prompt.accept()

    # Validate prompt result
    result = driver.find_element(By.ID, "result").text

    assert "Selenium Assignment 4" in result

    print("Prompt result:", result)

    # Keep browser open for screenshots
    time.sleep(10)

    # Close browser
    driver.quit()
```

Terminal

PS C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS> python -m pytest test_assignment4.py -v
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Rishita Saha\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
metadata: {'python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0', 'Plugins': {'html': '4.2.0', 'metadata': '3.1.1'}}
rootdir: C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS
plugins: html-4.2.0, metadata-3.1.1
collected 1 item

test_assignment4.py::test_javascript_alerts PASSED [100%]

===== 1 passed in 39.65s =====
PS C:\Users\Rishita Saha\Desktop\SELENIUM ASSIGNMENTS>

Successful Execution

Show the terminal after the script completes successfully.

7. Result

The JavaScript Alert was accepted, the Confirm Box was dismissed, and text was entered into the Prompt Box and submitted successfully using Selenium WebDriver.

8. Observation

- Selenium switches to JavaScript pop-ups using `driver.switch_to.alert`.
- `accept()` is used to confirm an alert.
- `dismiss()` is used to cancel a confirmation.
- `send_keys()` allows text to be entered into a Prompt Box.

9. Conclusion

The experiment successfully demonstrated how Selenium WebDriver can handle **JavaScript Alerts, Confirm Boxes, and Prompt Boxes** using the appropriate alert interaction methods.